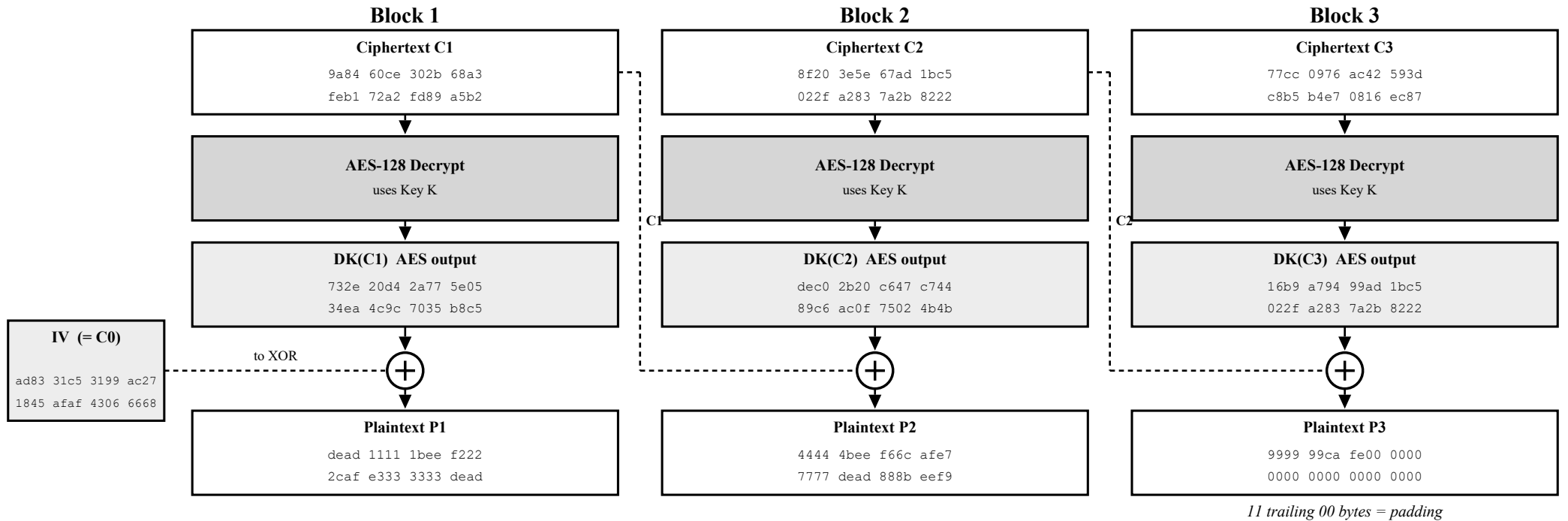


Part A: AES-128-CBC Decryption

Formula: $P_i = DK(C_i) \text{ XOR } C_{(i-1)}$, where $C_0 = IV$

Key K = 26671722 51915cc4 4cfc2ec3 a88b4d89



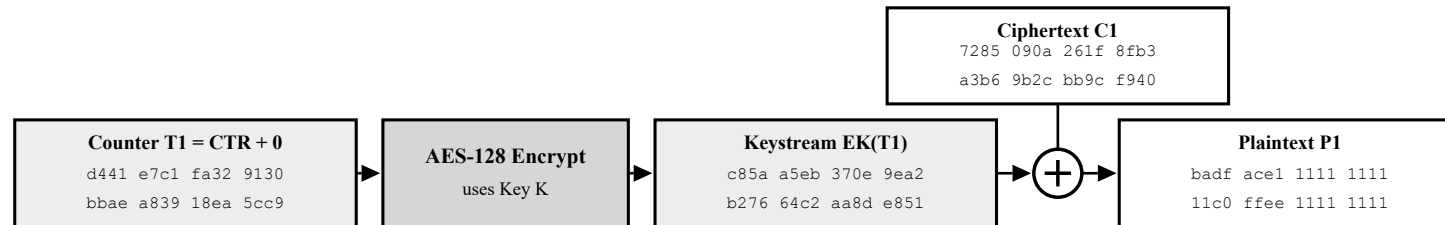
All values are hexadecimal. Chaining uses the previous ciphertext (or IV), not the previous plaintext.

Part B: AES-128-CTR Decryption

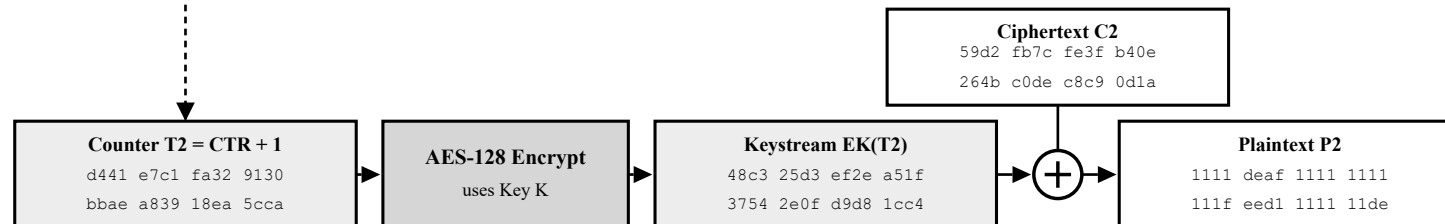
Formulas: $T_i = \text{CTR} + (i - 1)$ and $P_i = C_i \text{ XOR } EK(T_i)$

Key K = 53153913 a1623f8a 089ee173 60fb9ac9

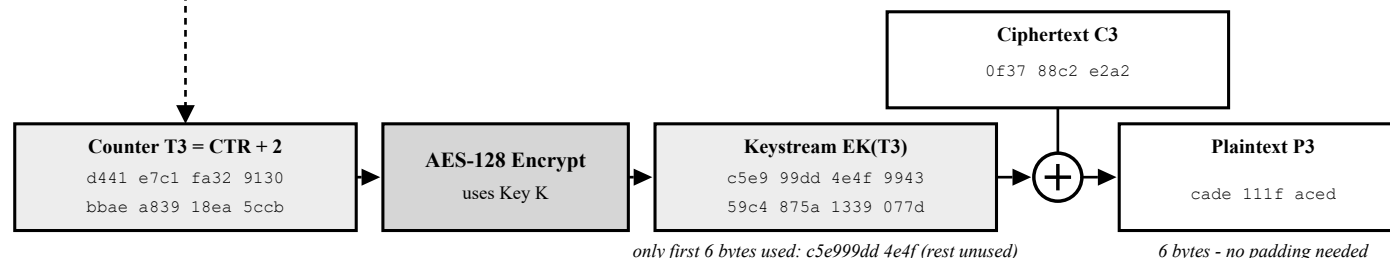
Block 1



Block 2



Block 3 (partial)



All values are hexadecimal. The counter increments by 1 for each block. Block 3 ciphertext is 6 bytes.

CS3002 - Assignment 2

Block Cipher Modes Decryption

Recovered Plaintexts and Discussion Answers

1. Recovered Plaintext - Message 1 (AES-128-CBC)

P1: dead1111 1beef222 2cafe333 3333dead
P2: 44444bee f66cafe7 7777dead 888beef9
P3: 999999ca fe000000 00000000 00000000

Full plaintext (48 bytes, hex):

dead1111 1beef222 2cafe333 3333dead 44444bee f66cafe7 7777dead 888beef9 999999ca
fe000000 00000000 00000000

The last eleven bytes are 00 padding. CBC requires full 16-byte blocks, so padding was added to complete the final block.

2. Question 4 - Can CBC Decryption Be Parallelized?

Yes. CBC encryption must run in order because each new ciphertext block needs the previous ciphertext. CBC decryption is different. All ciphertext blocks C1, C2 and C3 are already known as inputs. Each AES decrypt step $DK(C_i)$ only needs that block C_i and the shared key, so those AES operations can run at the same time. After that, XOR with IV or $C_{(i-1)}$ also uses only known ciphertext. The CBC diagram shows each AES decrypt box taking only its own C_i ; it does not wait for the previous plaintext. Therefore the sequential limit of CBC encryption does not apply to decryption in the same way.

3. Recovered Plaintext - Message 2 (AES-128-CTR)

P1: badface1 11111111 11c0ffee 11111111
P2: 1111deaf 11111111 111feed1 111111de
P3: cade 111f aced

Full plaintext (38 bytes, hex):

badface1 11111111 11c0ffee 11111111 1111deaf 11111111 111feed1 111111de cade111f aced

Length: 38 bytes. There are no padding bytes.

4. Question 6 - Why Is Padding Not Required in CTR Mode?

CTR mode turns AES into a stream cipher. It encrypts a counter to make a keystream, then XORs that keystream with the data. The ciphertext can be the same length as the plaintext. If the last piece is shorter than 16 bytes, only that many keystream bytes are used. In this message the last ciphertext is 0f3788c2e2a2 (6 bytes), so only the first 6 keystream bytes are XORed and the rest are unused. CBC works on full 16-byte blocks, so leftover space must be filled with padding (here trailing zero bytes on the CBC plaintext). CTR does not need that.

Method: AES-128 block encrypt/decrypt with CBC chaining and CTR counter increment, plus XOR.